

switch for Pattern Matching

Document #: P2940R0
Date: 2022-04-18
Project: Programming Language C++
SG17
Reply-to: Mihail Naydenov
<mihailnaydenov@gmail.com>

§ 1 Abstract

This paper argues, `switch` should be (re-)considered for Pattern Matching.

§ 2 Background

Current Pattern Matching (PM) approaches (p1371¹ + discussion,² p2392³) steer away from basing the PM upon the `switch`, extending it. This is done for two reasons.

- Technical difficulties in determining if an expression is C-style `switch` or a PM.
- Teachability concerns, because of perceived differences b/w `switch` and PM.

The first issue is undoubtedly valid, but is not unsolvable and a solution will be presented in the next section.

The second issue is more interesting. This paper argues `switch` *is already* PM.

There are simply few restrictions to apply in order to make a `switch`:

- the only pattern allowed, alongside the wildcard pattern, is an expression with integer result;
- all patterns must be consteval;
- all patterns must be OR-ed together;

```
auto some_value = 2;

switch (some_value) {
    case 1:
    case 2:
    case 3: // execute the same code
            for all 3
}

auto some_value = 2;

inspect (some_value) {
    1 OR // (placeholder OR syntax)
    2 OR
    3 => // execute the same code for
        all 3
};
```

PM now acts as a `switch`, working only on compile time integer expressions and will continue matching until all patterns are checked.

...This is... as long there is one statement to execute. As we know, classic switch can execute multiple cases, even these not matched, because of fallthrough. In PM, assuming OR functionality, we can have multiple patterns evaluated, but they will have to lead to the same code. As shown, this is similar to fallthrough with empty cases, with added bonus of being explicit about it.

Considering fallthrough with non-empty cases is, if not a bad practice, a bad default, it turns out PM is simply more feature rich and safer `switch`.

The fact, `switch` is really just a very limited PM, is recognized by other languages and some of them use it directly for PM (Swift, C#, Java). What is more, *all* languages recognize `switch` and PM are *not* different features and **no language has them both side by side!** Either `switch` is used from the get go (Swift), or `switch` is evolved to handle more general patterns (C#, Java) or a different spelling is used (`match`, `when`, etc).

If C++ introduces a separate construct while keeping `switch` it will be the only one do so. This will hardly ease teachability. A better approach would be to have two levels of the existing `switch`, old and new, much like we have already with `enum` and `enum class`.

An argument can also be made, we intend to make PM an expression, not a statement like `switch`. In practice however, a switch-like PM will be returning void, making the difference b/w it a the old switch only the need to use a semicolon after it.

OK, say for a moment, we have both the old `switch` and a PM system, called “inspect”. *When we will teach to use `switch`?* When we would say *don’t* use “inspect”? We can perform this kind of test with other control flow elements. For example, we can say, don’t use `if-else` chains, *because* and list few reasons. We can say, use `for over while`, *because* and we list again few good reasons.

Don’t use “inspect”, *because...*? And we really have no *good* reason, *one we would teach*. Sure, sometimes fallthrough with non-empty cases is “handy”, *but would we teach it?*

No. We would teach to keep using “inspect” and present options to solve the needs in a more structured way, like using recursion for example. `switch` will never be “the better tool for the job” simply because “inspect” effectively subsume it for all cases that matter.

Now, one can argue, we have a similar precedent in the face of `using` replacing `typedef`. We introduced a new, more powerful tool, “that does the same”, under a completely different name, and left the old name alone.

Even if we ignore the inherent problem of introducing “one more way to do the same thing”, the situation here is not the same. `using` introduced a new way to do a declaration (variable-like, left to right assignment), one that would eventually, naturally support template arguments. Achieving the same, reusing `typedef` would have been forced and contrived, even if possible syntactically. It is not the case here, we don’t need new form of anything. As you can see from the example above, both forms are almost identical, with further similarities explored in this paper. It is not like for example `switch`, compared to

`if-else` chain, where the difference in form is considerable (arbitrary number of queried items in a chain, vs one queried item in a tree).

We *could* have both side by side, but we *don't have to*. Especially, considering the painful process we'll have to come through when introducing a keyword with as popular spelling as “inspect” or “match”, that also can't be context aware my definition.

§ 3 Proposal

In the previous section we mentioned the parsing challenge of differentiating b/w regular `switch` and one that does PM, if we opt to reuse the introducer keyword.

This issue can be resolved by altering the syntax slightly and **instead of using round parenthesis, we use square ones**:

```
switch (a)      //< old C-switch for `a`
...
switch [a]      //< pattern match for `a`
...
switch [a, b]   //< pattern match for both `a` and `b`
```

With square brackets after `switch`, a PM expression is introduced, instead of a C-style `switch` statement.

This way we not only can continue to use the already reserved keyword, but also have a safe-by-default `switch`, one that does not fallthrough:

regular switch

```
auto some_value = 2;

switch (some_value) {
  case 1:
    ...
    break;
  case 2:
    ...
    // missed break
  case 3:
    ...
    break;
  ...
}
```

safer switch

```
auto some_value = 2;

switch [some_value] {
  case 1:
    ...
    break;
  case 2:
    ...
    // missed break, but no
    fallthrough
  case 3:
    ...
    break;
  ...
};
```

The only textual difference between these two are the brackets after `switch` and a semicolon at the end.

All functionality of `switch` remains the same, with the exception of fallthrough, in particular `default`, `break`, `return` and `continue` work exactly the same as they do

currently.

Because this is now PM, many new patterns are available. For example we can match strings:

```
auto some_value = string("hi");

switch [some_value] {
  case "hi":    // handle "hi"
  case "bye":   // handle "bye"
  default:     // handle all else
};
```

Or use advanced patterns:

```
auto some_value = Point(12, 13);

switch [some_value] {
  case [0, 0]: // handle point at origin
  case [0, _]: // handle x at origin
  case [_ , 0]: // handle y at origin
  ...
};
```

In other words, `switch` becomes full-featured PM, *with barely any new syntax*.

If we want the expression to have a result, we use a different case syntax and optionally a return type, as per current main proposal (p1371):

<pre>auto some_value = true; auto result = switch [some_value] { true => "yes"; false => "no"; };</pre>	<pre>auto some_value = true; auto result = switch [some_value] -> std::string { true => "yes"; false => "no"; };</pre>
--	--

`result` is `const char*`

`result` is `std::string`

Lastly, we can mix both case types.

We use `case:` and `default`, when we want a statement.

We use `=>` and `__`, when we want a result-producing expression:

<pre>enum class Op { Add, Sub, Mul, Div }; Op parseOp(Parser& parser) { return inspect (parser.consumeToken()) { '+' => Op::Add; '-' => Op::Sub; '*' => Op::Mul; '/' => Op::Div; let token => !{ std::cerr << "Unexpected: " << token; std::terminate(); } }; }</pre>	<pre>enum class Op { Add, Sub, Mul, Div }; Op parseOp(Parser& parser) { return switch [parser.consumeToken()] { '+' => Op::Add; '-' => Op::Sub; '*' => Op::Mul; '/' => Op::Div; case [[noreturn]] let token: { std::cerr << "Unexpected: " << token; std::terminate(); } }; }</pre>
--	---

A special `!{ }` block invented.

Reuse of case statements. Reuse of
`[[noreturn]]`.

This paper leaves many details out as they are already handled by P1371.

As you can see, evolving `switch` to handle PM is not only possible, but ultimately natural and beneficial, improving *both* the existing `switch` uses (safer, reacher) and the PM development as we can reuse its building blocks like introducer keyword, statement cases, etc.

Evolving `switch` also *keeps the language smaller*. There is less new syntax, less new ways of doing the same thing (!) and ultimately less new to learn for a newcomer.

If we have both PM and `switch`, which one should be thought first? Probably PM, the modern system. And this will have to be with something simple, so simple that will resemble `switch`. But at some point, one will have to learn `switch` as well, repeating the same process twice, once using the new form, once using the older, combined with a lesson why and how these two are different. **The more those two overlap, the less learning to be done.**

§ 3.0.1 But Wait, There is More!

This section is **not proposed**. It is here for possible future direction.

There is one more gift, `switch` can give us and this **reusing patterns outside PM!**

One problem every PM system has is that patterns always use *at least some* syntax that is already present in the language to mean something different.

This is not defect or deficiency, this is by design. In PM a pattern is declaration of expectation, which often matches the syntax of some real declaration or expression. This is *desirable*, this is what makes PM *natural*. In the regular language, when we say `0`, we

mean “create/set” `0`, in PM, when we say `0`, we mean “is it” `0`. This basic logic is *ideally* applied to all patterns, whenever possible, creating a syntax, which reuses the regular syntax in a different context.

Of course, this creates a problem. “Ideal” patterns can not be used in regular code, even if it is desirable - they already mean something else.

And that’s where `switch` again can be of help, because in `switch` **patterns have an introducer keyword** - `case`.

We only have to lift `case` out of `switch` and voila - we can have patterns inside regular code:

```
auto p = Point(12, 13);

if(case [_, let x] && [0,0] = p) { //< (using the Kona 2022 suggested
    // get x iff point is at origin
    syntax for the pattern)
}
```

The use of `case` here tells both us and the compiler, what follows is a pattern not regular code. This can change completely the meaning of the code:

```
auto p = Point(12, 13);
auto o = Point(0, 0);

if(case o = p) {
    // point is at origin, same as o == p
}
```

Without `case`, the expression would mean assign to `o` and test the assigned value, now it means “conditionally assign” where the condition is the pattern and the assignment itself is optional. (We opt not to assign in that example.)

Use of patterns inside `if` might not be to anyone’s liking because of the assignment inside `if` - something we had problems with for decades. Here however, `case` is a clear indicator, this `if` is different. This is considerable improvement, compared to any form, that might use patterns (+ assignment) directly inside `if`. Still, it might be easier to stomach an approach where not the assignment, but the test is prevalent. This is approach Kona 2022⁴ suggests, where a special, short form of PM exists, consisting only of introduction and patterns, no code. In the `switch` syntax it will look like that:

```
if(switch [p] [_, let x] && [0,0]) { //< (using the Kona 2022 suggested
    // get x iff point is at origin
    syntax for the patterns)
}
```

Notice, there is no explicit assignment inside `if`, the test is more visible due to the use of `switch`.

There is one big problem however - this can not be used *outside* `if`, like in regular assignments. In that scenario, only `case` can save us:

```
auto r = Rect(12, 13, 100, 200);  
  
case [let p, let [_, h]] = r;
```

This is an unconditional assignment where we use an advanced pattern to deconstruct the `rect`. `case` here ensures, the syntax is interpreted as a pattern.

In other words, no matter how much of the regular syntax we reuse, we are safe from ambiguity. For example we can invent a syntax, were we assign to an existing variable:

```
auto p = Point(12, 13);  
int x, y;  
  
case [&x, &y] = p;
```

`x` is now 12, `y` is now 13

Here we are *clearly* reusing syntax, yet we are safe to do so - it is in a well-defined different context.

Please note, a pattern `[x, y]` means “is equal to” a `Point(x, y)` as per p1371. We must have a different syntax.

The good news are, this new syntax **does not have to be unique to the language**.

As you can see, `switch` has some unique properties, that bring considerable value to PM. We should not let these go to waste.

-
1. Pattern Matching: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1371r3.pdf>↵
 2. Pattern Matching Discussion for Kona 2022: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2688r0.pdf>↵
 3. Pattern matching using `is` and `as`: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2392r2.pdf>↵
 4. Pattern Matching Discussion for Kona 2022: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2688r0.pdf>↵